

Enterprise AI Copilot for Cybersecurity

Complete Interview Question & Answer Handbook

Based on a production-grade Balbix-like AI chatbot architecture

Scope: Enterprise SaaS, AI Orchestration, RAG, Tool Calling, RBAC, Multi-Tenancy, Memory, Evaluation, Reliability, AWS Deployment

This document is designed for interview preparation. Each answer explains not only the concept, but also how the system behaves at runtime in a real enterprise production environment.

How to Use This Handbook

Use this as a principal-engineer-level interview preparation document. The goal is not to memorize every line, but to understand the design logic behind each answer.

The project is an enterprise AI copilot embedded inside a Balbix-like cybersecurity platform. It supports natural-language questions over live cyber-risk data, documents, reports, and approved workflows such as Jira ticket creation.

A strong answer should always connect business goal, runtime behavior, security boundary, failure handling, and tradeoff. This handbook follows that structure.

Coverage Map

1. Project Overview and Business Value
2. User Personas, RBAC, RequestContext, and Multi-Tenancy
3. High-Level Architecture and Request Lifecycle
4. AI Orchestrator, Intent Routing, Planning, and Agent Loop
5. Tool Execution Layer and Structured Data Access
6. RAG, PDF Processing, Vector Search, and Hybrid Retrieval
7. Conversation History, Redis, Durable Storage, and Long-Term Memory
8. Report Generation and Jira Workflow
9. Security Guardrails, DLP, Prompt Injection, and Hallucination Control
10. LLM Gateway, Model Routing, Prompt Design, and Fallbacks
11. Evaluation, Langfuse, Observability, and Quality Control
12. Reliability, Scalability, Caching, AWS Deployment, and Tradeoffs

Project Overview and Business Value

Q1. Explain the enterprise AI copilot project in simple terms.

The project is an enterprise AI copilot embedded inside a Balbix-like cybersecurity platform. It allows enterprise users such as CISOs, analysts, admins, and security teams to ask natural-language questions about cyber risk, assets, vulnerabilities, exposure, remediation, reports, and uploaded documents.

The important point is that this is not a generic chatbot. It is a secure agentic system that combines live structured data tools, RAG over documents, long-term user preferences, tenant-aware authorization, and production guardrails. The LLM does not directly access databases. It requests approved tools, receives authorized evidence, and then produces a grounded answer with citations or safe refusal.

The product value is faster investigation, easier executive reporting, better vulnerability prioritization, and reduced dashboard friction for enterprise customers.

Step-by-step runtime flow

- User opens the Balbix web app and authenticates through the existing product session.
- User asks a natural-language question, such as why risk increased this week.
- The system builds RequestContext with tenant, user, groups, permissions, and session metadata.
- The AI orchestrator decides whether to use live tools, RAG, memory, or a combination.
- The final answer is validated, cited, audited, and streamed back to the user.

Important components involved

- Balbix web UI
- Chat API and middleware
- RequestContext builder
- AI orchestrator
- Tool execution layer
- RAG service
- LLM gateway
- Policy engine
- Response validator
- Audit and observability pipeline

Different possible scenarios

- Simple product explanation uses product docs or generic knowledge.
- Risk or vulnerability questions use live tools.
- Policy or PDF questions use RAG.
- Mixed questions use both tools and RAG.
- Action questions, such as Jira creation, require approval.

Security, RBAC, and tenant-isolation considerations

- All data access is tenant-scoped.
- RBAC is enforced outside the LLM.
- The LLM only receives data already filtered by policy.
- Every answer that uses customer data must be traceable to tool results or document citations.

Production behavior: In production, this would be treated as a core product capability, not an experimental chat widget. It needs reliability, auditability, privacy controls, and enterprise-grade authorization from day one.

Q2. What business problem does this AI copilot solve for a cybersecurity platform?

Cybersecurity platforms generate huge amounts of data: asset inventory, vulnerabilities, exposure scores, business risk, remediation status, and trend dashboards. Users often know the question they want to ask but do not know which dashboard, filter, or report contains the answer.

The copilot reduces this operational friction. A CISO can ask for a board-level summary, while an analyst can ask for the top exploitable vulnerabilities affecting internet-facing assets. Instead of forcing users to manually join data across screens, the system orchestrates the right tools and explains the answer in context.

The measurable business outcomes are faster time-to-insight, better remediation prioritization, higher product adoption, fewer support requests, and improved customer trust in risk reporting.

Step-by-step runtime flow

- User expresses a business or security question in natural language.
- Intent router maps the query to a product capability such as risk summary, exposure analysis, or vulnerability prioritization.
- Tools fetch live platform data with authorization filters.
- RAG retrieves policy or documentation context if needed.
- The LLM converts raw results into an explanation suitable for the user persona.

Different possible scenarios

- CISO asks for business risk summary.
- Analyst asks for asset-level investigation.
- Admin asks for team-level remediation progress.
- Support user asks for product behavior explanation.

Edge cases and failure handling

- If the question is too vague, the bot asks clarification rather than guessing.
- If the required data source is unavailable, the bot returns a safe unavailable-data message.
- If the user lacks permission, the bot explains that access is restricted without leaking hidden data.

Production behavior: The system should be evaluated on task success, not just answer fluency. The real value is whether users can make faster and safer cybersecurity decisions.

Q3. Why is this not just a wrapper around an LLM API?

A simple LLM wrapper takes user text and sends it to a model. That is dangerous for enterprise cybersecurity because the system handles sensitive tenant data, asset records, vulnerabilities, risk scores, and customer documents.

This system wraps the LLM with an enterprise control plane. The LLM can reason and generate language, but it cannot directly query databases, ignore permissions, or execute actions. Every data access happens through approved tools and RAG services that enforce tenant isolation and RBAC.

The architecture separates intelligence from authority. The model proposes what it needs; the backend decides what is allowed; tools return facts; validators check the final answer.

Step-by-step runtime flow

- User query enters chat service.
- Orchestrator classifies intent and available tools.
- LLM may request a tool call.
- Policy engine validates the requested tool, arguments, and user permissions.
- Tool executes against authorized data only.

- LLM receives only safe tool output and generates the answer.
- Response validator checks for hallucination, unsupported claims, and sensitive data leakage.

Important components involved

- LLM gateway
- Policy engine
- Tool registry
- RAG retriever
- DLP scanner
- Response validator

Security, RBAC, and tenant-isolation considerations

- No raw database access by LLM.
- No cross-tenant data in prompts.
- Prompt injection cannot override backend authorization.
- Write actions require explicit approval and audit logs.

Production behavior: In an interview, emphasize this phrase: the LLM is not the security boundary. Backend policy, tools, and validators are the security boundary.

Q4. What are the main user personas and how do their needs differ?

The main personas are CISOs/CSOs, security analysts, vulnerability managers, tenant admins, developers, business risk owners, and customer success/support users. Each persona asks different questions and needs different answer styles.

A CISO usually wants high-level business risk, trend drivers, and board-ready summaries. A security analyst wants detailed assets, CVEs, exploitability, and remediation owners. A tenant admin may care about access, groups, and team progress. Customer success may need product documentation or customer-safe support explanations.

The system does not necessarily generate totally different truth for each role; the underlying facts remain the same. But answer style, available actions, and accessible data are controlled by role, group, and permissions.

Step-by-step runtime flow

- Authenticate user and determine role/group membership.
- Fetch relevant memories such as preferred response style.
- Use permissions to determine accessible data and allowed actions.
- Route the query to tools/RAG.
- Format the answer according to persona and user preference.

Different possible scenarios

- CISO asks for executive summary: answer emphasizes business impact and trends.
- Analyst asks for vulnerability details: answer includes assets, severity, exploitability, and remediation.
- User lacks access to a business unit: answer refuses or narrows scope.

Security, RBAC, and tenant-isolation considerations

- Persona should not override RBAC.
- A CISO-style answer still must only include authorized data.
- Support users must not see sensitive tenant data unless explicitly authorized.

Q5. What should be included in v1 and what should be kept out of scope?

For v1, I would include the web-app chatbot, authenticated access, RequestContext, RBAC-aware tools, RAG over controlled documents, conversation history, safe long-term memory for preferences, report generation, Jira ticket drafts with approval, audit logging, guardrails, and evaluation.

I would keep Slack/Teams, SOAR automation, automatic remediation, fine-tuning on customer data, and fully autonomous ticket creation out of scope. These are valuable, but they increase risk and integration complexity before the core security model is proven.

The v1 goal is not to do everything. It is to create a trusted, grounded, auditable copilot that enterprise customers can safely use inside the product.

Step-by-step runtime flow

- Define core user journeys.
- Build read-only intelligence first: risk, assets, vulnerabilities, RAG, reports.
- Add approval-based action for Jira.
- Instrument everything for tracing and evaluation.
- Expand integrations only after safety and quality gates are stable.

Different possible scenarios

- MVP demo can support limited tools.
- Enterprise beta should include RBAC, citations, audit logs, and safety.
- GA should include observability dashboards, eval pipelines, and customer admin controls.

Edge cases and failure handling

- Do not allow autonomous actions in v1.
- Do not store raw cyber facts in long-term memory.
- Do not let uploaded PDFs influence system instructions.

User Personas, RBAC, RequestContext, and Multi-Tenancy

Q6. How does authentication and RequestContext creation work?

When a user sends a message, the first production responsibility is not AI. It is identity. The middleware validates the existing product JWT or session token, extracts `user_id` and `tenant_id`, and then builds a trusted `RequestContext`.

`RequestContext` is the security envelope carried through the full AI flow. It contains `tenant_id`, `user_id`, `role`, `group_ids`, `permission_version`, allowed asset scopes, allowed document scopes, allowed actions, `request_id`, `session_id`, and `conversation_id`.

The LLM does not create or modify this context. It is built by backend services after authentication and authorization checks.

Step-by-step runtime flow

- Web UI sends chat request with JWT.
- API gateway/middleware validates token signature, expiry, issuer, and audience.
- Auth service fetches active user record, role, groups, permissions, and `permission_version`.
- `RequestContext` is created and attached to the internal request.
- Every downstream service receives `RequestContext` and enforces it.

Important components involved

- API gateway
- Auth middleware
- RBAC service
- Redis auth-context cache
- Chat service
- AI orchestrator

Edge cases and failure handling

- If JWT is expired, return 401 before calling AI.
- If RBAC service is down, fail closed for sensitive requests.
- If Redis cache is stale, rebuild from source of truth.
- If `permission_version` changed, invalidate cached context.

Security, RBAC, and tenant-isolation considerations

- `RequestContext` must be signed or trusted internally, not user-editable.
- Never accept `tenant_id` or `group_ids` directly from client body.
- Every tool must independently enforce `RequestContext`.

Q7. Why is group-based RBAC important for this system?

In a cybersecurity platform, users may not have access to every asset or business unit. A security analyst may be allowed to see Payments assets but not HR systems. Group-based RBAC lets the system restrict asset, vulnerability, document, and report access based on the user's assigned groups.

This matters because AI answers can accidentally aggregate or summarize hidden data. If the user asks "show all critical vulnerabilities," the tool should interpret that as all critical vulnerabilities within the user's allowed scope, not the whole tenant.

The LLM should never be responsible for this filtering. The database query, tool layer, RAG filter, and response validator enforce it.

Step-by-step runtime flow

- RequestContext includes group_ids and scoped asset groups.
- Tool planner selects a tool such as get_top_vulnerabilities.
- Policy engine checks permission can_read_vulnerabilities.
- Tool executes DB query with tenant_id and allowed group filters.
- Returned result contains only authorized records.
- Response validator checks cited records belong to the allowed scope.

Different possible scenarios

- User asks for a specific asset they can access: return details.
- User asks for a specific asset outside access: deny or say no accessible asset found.
- User asks for a tenant-wide summary but only has group scope: summarize only accessible scope and state that scope.

Security, RBAC, and tenant-isolation considerations

- Never reveal that a hidden asset exists if the user lacks access.
- Do not use memory to infer access.
- Cache keys must include tenant and permission_version.

Q8. How do you prevent cross-tenant data leakage?

Cross-tenant leakage is the most critical failure mode in a multi-tenant enterprise SaaS AI system. Prevention must be layered: database queries, vector search filters, Redis keys, document metadata, memory records, audit logs, and validators all include tenant_id.

The LLM should never receive data from another tenant. We do not rely on prompt instructions such as “do not reveal other tenants.” Instead, the backend retrieval systems are built so unauthorized data is never fetched.

Every source of context - messages, memories, tool results, chunks, reports, and Jira drafts - is tenant-scoped.

Step-by-step runtime flow

- RequestContext establishes tenant_id from trusted auth token.
- Tool calls include tenant_id in backend query filters.
- RAG queries filter chunks by tenant scope and document scope.
- Memory retrieval filters by tenant_id and user_id.
- Redis keys include tenant_id and conversation_id.
- Output validator verifies all citations and entity IDs belong to tenant_id.

Important components involved

- RBAC service
- Tool layer
- RAG metadata filters
- Memory service
- Redis
- Audit service
- Validator

Edge cases and failure handling

- If tenant_id is missing, reject request.
- If a tool returns mixed-tenant data, block response and raise critical alert.
- If vector metadata is incomplete, exclude the chunk.
- If cache key lacks tenant_id, treat as security bug.

Security, RBAC, and tenant-isolation considerations

- Fail closed on tenant ambiguity.
- Run automated tests for cross-tenant negative cases.
- Audit every access path.

Q9. How do you use Redis for auth context without creating stale permission risk?

Redis is useful for fast retrieval of user groups and permission context, but permissions can change. Therefore Redis is not the authority; it is a short-lived cache over the RBAC source of truth.

The cached auth context should include `permission_version`. When an admin changes user permissions, the version increments. On the next request, if Redis has an older version, the system rebuilds `RequestContext` from the RBAC service.

Use short TTLs, such as two to five minutes, and event-driven invalidation if available.

Step-by-step runtime flow

- Middleware extracts `user_id` and `tenant_id`.
- Check Redis key `auth_context:{tenant_id}:{user_id}`.
- If cache hit and `permission_version` is current, use it.
- If cache miss/stale, fetch from RBAC DB/service.
- Store refreshed context in Redis with TTL.
- Pass `RequestContext` downstream.

Edge cases and failure handling

- If Redis fails, fetch from RBAC DB.
- If RBAC DB fails and cache is missing, fail closed.
- If cache exists but is expired, rebuild.
- If permissions changed during a long workflow, re-check before action execution.

Security, RBAC, and tenant-isolation considerations

- Do not cache broad admin permissions longer than necessary.
- Do not trust `group_ids` sent from frontend.
- Use tenant-aware cache keys.

Q10. How should unauthorized user questions be handled?

Unauthorized questions should be handled carefully. The system should not leak whether a restricted asset, vulnerability, document, or business unit exists. It should respond with a safe access-limited message or provide results only from the user's authorized scope.

For example, if the user asks for "all crown-jewel assets" but only has access to one group, the system should either answer for the accessible group or say it cannot access data outside the user's permission scope.

The refusal should be professional and not reveal hidden metadata.

Step-by-step runtime flow

- Intent router identifies requested resource or scope.
- Policy engine checks requested scope against `RequestContext`.
- If partially allowed, narrow query to accessible scope.
- If fully denied, return safe denial.
- Audit the denied access attempt.
- Do not call LLM with unauthorized data.

Different possible scenarios

- Fully allowed: answer normally.
- Partially allowed: answer with scope clarification.
- Fully unauthorized: deny safely.
- Ambiguous scope: ask clarification or default to authorized scope.

Security, RBAC, and tenant-isolation considerations

- Do not say “HR asset exists but you cannot access it.”
- Do not include hidden counts.
- Log denied attempts for security monitoring.

High-Level Architecture and Request Lifecycle

Q11. Describe the high-level architecture of the chatbot.

The high-level architecture has a web UI, API gateway, chat service, RequestContext builder, AI orchestrator, tool execution layer, RAG service, memory service, LLM gateway, policy engine, response validator, audit service, and observability pipeline.

The key design is that the AI orchestrator coordinates decisions, but does not own trust. Tools provide live facts. RAG provides document evidence. The policy engine controls access. The validator checks the answer before the user sees it.

This separation makes the system scalable, testable, and secure.

Step-by-step runtime flow

- User sends query from Balbix web app.
- Chat API authenticates and builds RequestContext.
- Conversation context and memory are loaded.
- Intent router/planner creates an execution plan.
- Policy engine approves or rejects each step.
- Tools/RAG fetch evidence.
- LLM generates answer using authorized evidence.
- Validator checks the response.
- Answer streams to UI and logs are stored.

Important components involved

- Frontend chat UI
- API gateway
- Chat service
- AI orchestrator
- Tool registry
- RAG service
- Vector DB
- LLM gateway
- Policy engine
- Memory service
- Audit/observability

Production behavior: This should be deployed as independently scalable services, with async workers for long-running reports and document ingestion.

Q12. What happens when a user sends a chat message?

A production chat request is not a single LLM call. It is a controlled execution pipeline. The system first authenticates the user, persists the message, loads context, decides what kind of work is needed, executes authorized retrieval/tool steps, generates the answer, validates it, streams it, and logs the full trace.

The behavior changes based on the user's question. A simple product question may require only RAG or direct explanation. A live risk question requires tools. A policy-comparison question requires RAG plus tools. A Jira request requires approval. An ambiguous query requires clarification.

The system should always prefer safe, grounded behavior over fast but unsupported answers.

Step-by-step runtime flow

- Frontend sends POST /chat/message with JWT, conversation_id, and message.
- Middleware validates JWT and creates RequestContext.
- Chat service writes the user message to durable DB with sequence number and status RECEIVED.
- Redis recent context is updated best-effort; if it fails, DB remains source of truth.
- AI orchestrator loads recent messages, conversation summary, and relevant user memories.
- Intent router identifies whether the query needs tools, RAG, clarification, async workflow, or action approval.
- Planner decomposes multi-part requests into a task graph.
- Policy engine validates each planned task before execution.
- Tools and RAG execute with tenant/RBAC filters.
- Agent loop may call additional tools if initial results show more data is needed.
- LLM generates final answer from authorized evidence.
- Response validator checks citations, RBAC, unsupported claims, and DLP.
- Answer is streamed to UI over SSE.
- Final assistant message, tool calls, citations, latency, cost, and audit events are persisted.
- Memory extraction runs asynchronously after response completion.

Important components involved

- Chat API
- RequestContext builder
- Conversation store
- Redis context cache
- AI orchestrator
- Planner
- Policy engine
- Tool layer
- RAG service
- LLM gateway
- Validator
- Audit service

Different possible scenarios

- Simple informational question: answer directly or from docs.
- Document question: RAG retrieves allowed chunks.
- Live data question: tool fetches current data.
- Mixed question: RAG extracts rules and tools fetch live records.
- Unauthorized question: policy denies or narrows scope.
- Tool timeout: return partial or safe unavailable-data response.
- Ambiguous question: ask clarification.

Edge cases and failure handling

- If DB write fails at request start, do not proceed because audit/history would be broken.
- If Redis fails, continue and repair cache later.
- If LLM provider fails, retry then fallback to backup model.
- If validator fails, regenerate with stricter prompt or return safe fallback.
- If user cancels stream, persist cancelled status.

Security, RBAC, and tenant-isolation considerations

- Every retrieved record must match tenant_id and user scope.
- Retrieved documents are evidence, not instructions.

- No final answer is sent until validation passes for sensitive queries.

Production behavior: A strong interview answer should show this as a pipeline with safety gates, not as “send prompt to LLM and return response.”

Q13. How does the system decide whether a query is synchronous or asynchronous?

The orchestrator estimates the complexity of the task. Simple Q&A, small tool calls, and short RAG retrieval can run synchronously with streaming. Heavy workflows such as report generation, large PDF analysis, multiple business-unit comparisons, and bulk Jira ticket creation should run asynchronously.

This prevents timeouts and creates a better user experience. The chat can immediately acknowledge the job, show progress, and later provide a download link or result summary.

A production system should have explicit thresholds for tool count, expected latency, document size, and action risk.

Step-by-step runtime flow

- Planner decomposes the user request.
- Estimator calculates expected work: tool calls, data size, PDF size, report rendering, external actions.
- If within sync limits, execute in chat loop.
- If above limits, create workflow job and return job status.
- Worker executes the workflow with retries and checkpoints.
- User receives progress and final artifact.

Different possible scenarios

- Top 10 assets: synchronous.
- Generate monthly board report: asynchronous.
- Analyze 200-page PDF: asynchronous.
- Create 100 Jira tickets: async plus approval and batching.

Edge cases and failure handling

- If sync workflow exceeds budget, stop and offer async continuation.
- If async job fails, preserve failure reason and allow retry.
- If user disconnects, job continues only if explicitly accepted.

Q14. How should streaming responses work?

Streaming improves perceived latency. The backend can stream the final answer tokens to the UI while the model generates. However, streaming must be handled carefully because partial text might later fail validation.

For sensitive answers, a safer pattern is to stream status updates first, then stream final answer only after tool/RAG steps are complete and final generation begins. For high-risk actions or sensitive data, the system may buffer the final answer until validation passes.

Streaming state can be stored temporarily in Redis, while the final completed assistant message is written to durable DB.

Step-by-step runtime flow

- Frontend opens SSE stream or receives streaming response.
- Backend sends status events such as analyzing, retrieving data, calling tools.
- After evidence is ready, LLM generation begins.
- Tokens are streamed to UI.
- Partial stream state is stored in Redis.
- On completion, final answer is persisted to DB.

- If validation fails, stream is replaced with safe fallback or regenerated before exposure depending on risk mode.

Important components involved

- SSE endpoint
- Chat service
- Redis streaming state
- LLM gateway
- Validator
- Conversation DB

Edge cases and failure handling

- User cancels stream: mark assistant message CANCELLED.
- Network disconnect: allow resume using message_id if final answer completed.
- LLM error mid-stream: show safe error and persist FAILED status.
- Validator failure: do not persist unsafe answer as final.

Security, RBAC, and tenant-isolation considerations

- Do not stream raw tool output directly.
- Do not stream unvalidated secrets or cross-tenant data.
- Use buffered mode for high-risk responses.

AI Orchestrator, Intent Routing, Planning, and Agent Loop

Q15. What is the responsibility of the AI orchestrator?

The AI orchestrator is the coordination brain of the system. It does not own raw data or permissions; instead, it coordinates memory retrieval, intent classification, task planning, policy checks, tool execution, RAG retrieval, LLM calls, response validation, and post-response tasks.

In production, the orchestrator is a state machine or workflow controller. It tracks working state, completed steps, remaining tasks, tool results, and stop conditions. This makes the agent behavior bounded and debuggable.

The orchestrator is the difference between a simple chatbot and a production AI agent system.

Step-by-step runtime flow

- Receive user_message and RequestContext.
- Load conversation and relevant memories.
- Classify intent and identify task type.
- Create plan or task graph.
- Run policy check for planned steps.
- Execute tools/RAG with retries and timeouts.
- Update working state.
- Call LLM for final answer or next tool decision.
- Validate and return response.
- Trigger audit and memory extraction.

Important components involved

- Intent router
- Task decomposer
- Planner
- Loop controller
- Tool executor
- RAG retriever
- LLM gateway
- Validator

Production behavior: The orchestrator should emit structured traces for every internal step, otherwise debugging production failures becomes almost impossible.

Q16. How does intent detection work?

Intent detection classifies what the user wants before executing expensive or sensitive steps. It determines whether the query is general Q&A, risk summary, asset query, vulnerability query, exposure analysis, historical trend, document Q&A, report generation, Jira action, follow-up, or clarification-needed.

Intent detection can be implemented with a small LLM, a classifier, rules, or hybrid logic. In production, I prefer hybrid: rules for obvious commands and a small model for ambiguous natural language.

The output should be structured and machine-readable, not free text.

Step-by-step runtime flow

- Normalize user query and include conversation context.
- Run lightweight classifier or rules.

- Return structured intent with confidence.
- If confidence is low, ask clarification or route to planner with safe defaults.
- Attach intent to trace for evaluation.

Different possible scenarios

- “Show top risky assets” maps to ASSET_QUERY.
- “What does this PDF say?” maps to DOCUMENT_QA.
- “Create tickets” maps to ACTION_REQUEST.
- “Do the same for Payments” maps to FOLLOW_UP needing conversation context.

Edge cases and failure handling

- If multiple intents are detected, call task decomposer.
- If intent conflicts with permissions, policy engine decides.
- If query is vague, ask clarification instead of guessing.

Security, RBAC, and tenant-isolation considerations

- Intent classification does not grant permission.
- Even if intent says Jira creation, action still requires approval and user permission.

Q17. How do you handle multiple questions in one user message?

Users often ask compound questions. For example: “Analyze this PDF, compare it with current vulnerabilities, generate a report, and create Jira tickets for top issues.” This cannot be treated as one intent. It must be decomposed into tasks.

The task decomposer extracts individual tasks, and the workflow planner builds a dependency graph. Independent tasks can run in parallel. Dependent tasks must run sequentially. Risky actions pause for approval.

This makes the system reliable and avoids blindly sending a complex instruction to the LLM.

Step-by-step runtime flow

- Task decomposer extracts tasks from user message.
- Each task gets intent, inputs, dependencies, and risk level.
- Planner creates a DAG.
- Policy engine validates each task.
- Executor runs independent tasks in parallel and dependent tasks sequentially.
- LLM combines results into one coherent answer.
- Actions pause for approval.

Important components involved

- Task decomposer
- Workflow planner
- DAG executor
- Policy engine
- Tool/RAG services

Different possible scenarios

- Independent: “show top assets and risk trend” can run parallel.
- Sequential: “analyze PDF then compare with vulnerabilities” must run in order.
- Action: “create Jira tickets” requires approval after draft preview.

Edge cases and failure handling

- If one task fails, return partial result for successful tasks.

- If task graph is too large, convert to async workflow.
- If dependencies are unclear, ask clarification.

Q18. How does the agent loop work when the LLM asks for tools repeatedly?

The system uses a bounded agent loop. The LLM can decide it needs a tool, observe the result, and then decide whether it needs another tool. However, the loop is controlled by the orchestrator, policy engine, and stop conditions.

For example, to answer “why did risk increase this week,” the LLM may first request risk trend, then risk drivers, then top affected assets. Each tool call is validated before execution.

This is agentic behavior, but not autonomous uncontrolled behavior.

Step-by-step runtime flow

- LLM receives query, working state, available tool descriptions, and evidence so far.
- LLM emits structured decision: `final_answer`, `tool_call`, `clarification`, or `async_needed`.
- Tool request parser validates schema.
- Policy engine checks permission and risk.
- Tool executes and output is validated.
- Working state is updated.
- Loop repeats until final answer or stop condition.

Important components involved

- Loop controller
- LLM gateway
- Tool request parser
- Policy engine
- Tool executor
- Working state store

Edge cases and failure handling

- Max iterations reached: return partial answer or async handoff.
- Tool denied: explain access restriction safely.
- Tool timeout: retry or return unavailable-data message.
- Repeated same tool call: detect loop and stop.

Security, RBAC, and tenant-isolation considerations

- The LLM requests tools; it does not execute them.
- Policy is checked before every tool call.
- Tool output is validated before being shown to LLM.

Q19. What are the stop conditions for the agent loop?

Stop conditions prevent infinite loops, excessive cost, latency spikes, and unsafe behavior. The orchestrator must know when to stop asking the LLM to reason and either answer, ask clarification, return partial results, or create an async job.

Common limits include max iterations, max tool calls, max wall-clock time, max token budget, and max parallel tool calls. There are also semantic stop conditions such as approval required or missing data.

Stop conditions should be enforced by backend logic, not by prompt instructions alone.

Step-by-step runtime flow

- Before loop starts, initialize budgets and limits.

- After each LLM/tool step, update counters and elapsed time.
- Check if final answer is ready.
- Check if clarification or approval is required.
- Check if limits are exceeded.
- Stop and return best safe outcome.

Edge cases and failure handling

- If max iterations hit, summarize findings and say deeper analysis needs async workflow.
- If tool results are insufficient, say evidence is incomplete.
- If user approval is needed, pause before action.
- If policy denies access, stop immediately.

Production behavior: Track stop-condition metrics in observability. Frequent max-iteration exits usually mean prompts, tool descriptions, or planner logic need improvement.

Q20. How do you handle follow-up questions like “do the same for Payments”?

Follow-up questions depend on conversation context. The system must resolve references like “same,” “this,” “those assets,” or “last report.” It should use recent messages, conversation summary, and working state references, but still fetch live data again when facts are needed.

For example, if the previous query generated a risk summary for HR and the user says “do the same for Payments,” the planner should reuse the previous intent and report format but change the business-unit filter to Payments.

Memory can influence style, but live tools still provide current facts.

Step-by-step runtime flow

- Load last N messages and conversation summary.
- Resolve pronouns and references.
- Identify previous task structure and filters.
- Apply user’s new constraint, such as Payments.
- Re-run tools/RAG with current RequestContext.
- Generate answer in the same style if appropriate.

Edge cases and failure handling

- If reference is unclear, ask clarification.
- If previous context is missing from Redis, reload from durable DB.
- If user no longer has permission, deny or narrow scope.
- If data changed since last answer, use current tool results and mention timestamp.

Security, RBAC, and tenant-isolation considerations

- Do not reuse previous sensitive tool outputs if permissions changed.
- Do not use memory as source of truth for cyber data.

Q21. How should vague or ambiguous questions be handled?

Ambiguous questions should not be answered with fabricated assumptions, especially in cybersecurity. If a user asks “What are our top risks?” the system must decide whether this means business risk, vulnerability risk, asset exposure, compliance risk, or cyber risk quantification.

The system can either ask a clarification question or provide a safe default with clearly stated scope. For enterprise risk, I prefer asking clarification when multiple interpretations could materially change the answer.

This improves trust and prevents wrong decisions.

Step-by-step runtime flow

- Intent classifier detects low confidence or multiple possible intents.
- Planner checks whether a default product interpretation exists.
- If ambiguity affects data/action scope, ask clarification.
- If ambiguity is minor, answer with stated assumptions.
- Log clarification event for evaluation.

Different possible scenarios

- “Top risks” asks clarification between CRQ, vulnerabilities, exposure, compliance.
- “Generate report” asks what report type or time range.
- “Create tickets” asks which findings and confirms approval.

Edge cases and failure handling

- Do not ask unnecessary clarification if context clearly resolves the query.
- Do not default to tenant-wide scope if user has limited access.
- Do not perform write actions on ambiguous requests.

Tool Execution Layer and Structured Data Access

Q22. What is the tool execution layer and why is it needed?

The tool execution layer is the safe bridge between the AI orchestrator and Balbix production data. The LLM can request a tool, but the backend validates, authorizes, executes, and audits the call.

This layer prevents the LLM from directly querying databases or calling arbitrary APIs. Each tool has a schema, permission requirements, timeout, allowed argument ranges, output contract, and audit logging.

The tool layer turns AI intent into safe backend operations.

Step-by-step runtime flow

- LLM emits structured tool request.
- Tool request parser checks tool name and JSON schema.
- Policy engine checks RequestContext and required permission.
- Tool executor applies tenant/RBAC filters and calls backend service/DB.
- Output validator checks result shape, tenant IDs, data size, and secrets.
- Result is added to working state and passed to LLM.

Important components involved

- Tool registry
- Policy engine
- Schema validator
- Tool executor
- Output validator
- Audit logger

Edge cases and failure handling

- Invalid tool name: reject and ask LLM to choose valid tool.
- Invalid args: repair if safe or ask clarification.
- Tool timeout: retry with backoff or return safe error.
- Oversized result: summarize/aggregate before LLM.

Security, RBAC, and tenant-isolation considerations

- Tools enforce tenant and RBAC in queries.
- LLM never receives unauthorized data.
- Write tools require approval and idempotency.

Q23. What tools would you expose for a Balbix-like AI copilot?

I would expose business-level tools, not raw database access. The tools should align with product capabilities: risk summary, asset search, vulnerability prioritization, exposure analysis, historical trends, report generation, and Jira draft creation.

Each tool should return structured JSON with data_timestamp, result rows, summaries, citations, and links back to the product. The output should be compact enough for LLM use, not a raw dump of thousands of rows.

Tool names should be stable and auditable.

Step-by-step runtime flow

- Planner selects tool based on intent.

- Policy engine verifies user permission.
- Tool executes with scoped filters.
- Tool returns summarized structured output.
- LLM uses output to explain findings.

Important components involved

- get_current_risk_summary
- get_risk_trend
- get_top_risk_drivers
- get_top_risky_assets
- search_assets
- get_asset_details
- get_top_vulnerabilities
- get_exposure_summary
- generate_report_job
- prepare_jira_ticket

Different possible scenarios

- Risk query uses risk tools.
- Asset query uses asset tools.
- Vulnerability query uses vulnerability tools.
- Report query uses report service.
- Jira request uses draft/approval/action tools.

Security, RBAC, and tenant-isolation considerations

- Do not expose generic SQL execution tool.
- Do not expose admin-only tools to normal users.
- Tool availability should be filtered by RequestContext.

Q24. Why are tools better than RAG for live structured data?

Live product data such as assets, vulnerabilities, risk scores, and remediation status is structured, permission-sensitive, and frequently changing. Vector search is not ideal for exact filtering, aggregation, sorting, and freshness.

A tool can run deterministic SQL or service calls with tenant filters, group filters, time windows, and product-specific ranking logic. It can return current data with a timestamp.

RAG is better for documents and policies. Tools are better for source-of-truth product data.

Step-by-step runtime flow

- User asks live data question.
- Intent router marks tools_needed=true.
- Tool applies tenant/group filters.
- DB/analytics store returns current data.
- Tool summarizes and returns timestamped result.
- LLM explains result with citations.

Different possible scenarios

- “Top 10 risky assets” uses tool.
- “Risk trend over 90 days” uses analytics tool.
- “What does policy say?” uses RAG.
- “Which vulnerabilities violate policy?” uses both.

Edge cases and failure handling

- If data freshness is important, answer includes last updated timestamp.
- If tool result conflicts with old document, prefer live tool and mention document age if relevant.

Q25. How do you validate tool inputs and outputs?

Tool validation happens before and after execution. Input validation ensures the LLM requested a real tool with allowed arguments. Output validation ensures the tool returned safe, scoped, structured data.

This is important because LLM-generated tool arguments may be malformed, overly broad, or unauthorized. Similarly, backend service bugs can accidentally return excessive or cross-tenant data.

Validation is a production safety gate.

Step-by-step runtime flow

- Parse tool request as JSON.
- Validate against tool schema.
- Normalize safe fields such as time range or limit.
- Check policy and permissions.
- Execute tool.
- Validate output fields, tenant scope, result size, and sensitive data.
- Attach result to trace.

Edge cases and failure handling

- limit too high: clamp to allowed max.
- business_unit not in scope: deny or ask clarification.
- tool returns missing timestamp: mark low confidence or fail validation.
- result contains secret: redact/block before LLM.

Security, RBAC, and tenant-isolation considerations

- Never pass unvalidated tool output to LLM.
- Validate cited asset/document IDs against RequestContext.
- Log validation failures as security or reliability events.

Q26. How do you handle large tool results?

The LLM should not receive huge raw datasets. For example, sending 50,000 vulnerabilities to the model is expensive, slow, and risky. The tool layer should aggregate, rank, paginate, and summarize before returning evidence to the LLM.

The backend should perform deterministic filtering and computation. The LLM should only explain the compact result and provide links to drill down in the product UI.

This approach improves latency, cost, reliability, and privacy.

Step-by-step runtime flow

- Tool receives query with filters and limit.
- Backend applies filters and ranking.
- If result set is large, compute aggregates and top examples.
- Return summary, counts, top N rows, and drill-down link.
- LLM generates explanation from compact evidence.

Different possible scenarios

- Top 10 view: return rows.

- Thousands of records: return counts, categories, and export link.
- Report generation: async job can process larger data outside prompt.

Edge cases and failure handling

- If user asks for all records, provide filtered export workflow rather than LLM dump.
- If data exceeds threshold, switch to async report/export.
- If aggregation fails, return safe partial summary.

Q27. How do you handle historical trend questions?

Historical questions such as “how did risk change in the last 90 days?” should use analytics or time-series tools, not memory or RAG. The tool should query historical snapshots or event tables and compute deltas.

The answer should explain trend direction, key drivers, affected business units, and data freshness. If the user asks “why,” the system may call additional tools for risk drivers, new vulnerabilities, exposure changes, or asset changes.

The agent loop is useful here because one tool result may reveal the need for more detail.

Step-by-step runtime flow

- Intent router identifies HISTORICAL_TREND.
- Tool fetches risk score time series for allowed scope.
- If delta is significant, planner calls driver-analysis tool.
- Tool fetches top changed assets/vulnerabilities/exposures.
- LLM explains the trend with timestamps and citations.

Edge cases and failure handling

- Missing historical data: state that historical data is unavailable for that range.
- Large time range: aggregate by week/month.
- Permission changes: query only currently allowed scope unless product requires historical permission model.

Security, RBAC, and tenant-isolation considerations

- Do not use previous chat memory as historical truth.
- Do not compare against data outside user’s accessible scope.

RAG, PDF Processing, Vector Search, and Hybrid Retrieval

Q28. Describe the RAG architecture for documents and PDFs.

RAG has two separate flows: ingestion and query-time retrieval. During ingestion, documents are uploaded, stored, parsed, chunked, embedded, indexed, and assigned metadata. During query time, the user query is embedded and searched against only the documents the user is allowed to access.

The most important design requirement is permission-aware retrieval. Every chunk carries tenant_id, document scope, allowed groups, owner, page number, document type, and classification.

The LLM sees only safe, filtered, relevant chunks as evidence.

Step-by-step runtime flow

- Document uploaded through UI or admin/customer-success flow.
- Original file stored in S3 with KMS encryption.
- Metadata stored in DB.
- Parser extracts text, headings, tables, and page numbers.
- Chunker creates section-aware chunks.
- Embedding service generates vectors.
- Vector DB stores embeddings with metadata.
- At query time, retriever applies tenant/group/user/session filters.
- Reranker selects top chunks.
- DLP and prompt-injection scans run before LLM.

Important components involved

- S3
- Document metadata DB
- Parser/OCR service
- Chunker
- Embedding service
- Vector DB/OpenSearch
- Reranker
- RAG service
- DLP scanner

Security, RBAC, and tenant-isolation considerations

- Documents are evidence, not instructions.
- Chunks without complete permission metadata are excluded.
- Session uploads are visible only in the current session/user scope.

Q29. How do you handle uploaded PDFs in the current chat?

When a user uploads a PDF in the current chat, the system treats it as a session-scoped or user-private document. It should not automatically become available to the whole tenant unless the uploader has permission and explicitly chooses that scope.

For small PDFs, ingestion can be synchronous. For larger or scanned PDFs, ingestion should be asynchronous with status updates. The user can ask questions only after the document is ready or receive a message that processing is still happening.

The document metadata must include `tenant_id`, `uploader_id`, `conversation_id`, `document_scope`, `processing_status`, and page-level citation support.

Step-by-step runtime flow

- User uploads PDF.
- API validates file type, size, malware scan, and upload permission.
- Store original PDF in S3 with KMS.
- Create document record with status `PROCESSING` and scope `SESSION_DOC` or `USER_DOC`.
- Parse text/tables/OCR if needed.
- Chunk and embed content.
- Index chunks with metadata.
- Mark status `READY`.
- RAG retrieval can now use the document for that user/session.

Edge cases and failure handling

- Malformed PDF: mark `FAILED` and show safe error.
- Large PDF: async processing with progress.
- Scanned PDF: OCR or multimodal extraction.
- Prompt injection text inside PDF: flagged and treated as untrusted content.

Security, RBAC, and tenant-isolation considerations

- Do not make session upload tenant-wide by default.
- Run malware scanning before processing.
- Apply DLP before sending chunks to LLM.

Q30. How should PDF chunking be done?

PDF chunking should be document-aware, not arbitrary fixed-size splitting. Cybersecurity documents often have headings, tables, controls, policies, and remediation SLAs. Splitting blindly can separate important context and reduce retrieval quality.

A good chunk contains a coherent section, its heading path, page number, neighboring context, and source metadata. Tables should be converted into readable text or structured markdown so that retrieval can understand them.

Chunk size should balance retrieval quality and token cost.

Step-by-step runtime flow

- Extract pages with layout information.
- Detect headings, subheadings, tables, and section boundaries.
- Create chunks by semantic section.
- Attach metadata: `document_id`, page number, section path, `tenant_id`, scope.
- Create embeddings.
- Store in vector DB and metadata DB.

Different possible scenarios

- Policy paragraph chunk includes section heading.
- Table chunk includes row meanings in text.
- Long section is split with overlap.
- Scanned pages use OCR and confidence scores.

Edge cases and failure handling

- If OCR confidence is low, mark chunk quality low.

- If table extraction fails, store page reference and fallback text.
- If chunk contains secrets, redact or exclude.

Q31. Why use hybrid search instead of only vector search?

Cybersecurity queries often contain exact identifiers such as CVE IDs, asset names, control IDs, hostnames, business units, and policy names. Pure vector search may miss exact matches. Hybrid search combines semantic similarity with keyword/exact matching.

For example, a query for CVE-2025-1234 should match the exact CVE string, not a semantically similar vulnerability. At the same time, a query like “ransomware exposure policy” benefits from semantic search.

Hybrid search improves recall and precision.

Step-by-step runtime flow

- Generate embedding for user query.
- Run vector search with permission filters.
- Run keyword/BM25 search with same filters.
- Merge candidates using weighted scoring.
- Rerank top candidates.
- Return top evidence chunks with citations.

Different possible scenarios

- Exact CVE query: keyword dominates.
- Conceptual policy query: vector dominates.
- Mixed query: both contribute.
- Asset ID query: exact filter can be used.

Edge cases and failure handling

- If hybrid results conflict, reranker and source metadata decide.
- If exact match found in unauthorized doc, exclude it.
- If no result found, say insufficient document evidence.

Q32. How do you enforce permissions in vector search?

Every vector chunk must include authorization metadata. Retrieval applies filters before or during search so that unauthorized chunks are never returned. The filter includes tenant_id, document scope, allowed group IDs, uploader ID, and session/conversation ID.

This is critical because vector similarity alone does not understand enterprise permissions. Permission-aware retrieval must be implemented in the RAG service and vector DB query, not left to the LLM.

If metadata is missing or ambiguous, the chunk should be excluded.

Step-by-step runtime flow

- RAG service receives RequestContext.
- Build metadata filter from tenant_id, group_ids, user_id, conversation_id, and allowed_doc_scopes.
- Run vector/keyword search with filters.
- Rerank only allowed candidates.
- Validate returned chunks again before prompt construction.

Different possible scenarios

- Global doc: available to all tenants if truly generic.

- Tenant doc: same tenant only.
- Group doc: group overlap required.
- User doc: uploader or allowed user only.
- Session doc: same conversation/session only.

Security, RBAC, and tenant-isolation considerations

- Do not use one unfiltered global vector search.
- Do not rely on post-generation redaction.
- All chunk citations must be permission-checked.

Q33. What happens if retrieved document data conflicts with live tool output?

Tool output from live Balbix data should generally be treated as source of truth for current operational facts. Documents may contain policies, templates, explanations, or older reports. If a document says there are 10 critical vulnerabilities but the live tool says there are 14, the final answer should prefer the live tool for current count and cite the document only as historical or policy context.

The system should not silently merge conflicting facts. It should explain the conflict and include timestamps when available.

This distinction is important for trust.

Step-by-step runtime flow

- RAG retrieves document evidence.
- Tool fetches live structured data.
- Evidence fusion layer labels sources and timestamps.
- Conflict detector compares overlapping claims.
- LLM is instructed to prefer live tool data for current facts.
- Final answer explains discrepancy if relevant.

Different possible scenarios

- Policy rule from document + live vulnerability data: both are valid.
- Old report count conflicts with current tool: prefer current tool.
- Two documents conflict: cite both and ask clarification or use latest approved doc.

Edge cases and failure handling

- If source freshness is unknown, mention uncertainty.
- If conflict affects action, do not create Jira until clarified.
- If conflict is severe, log quality issue.

Q34. How do you prevent prompt injection through retrieved documents?

Retrieved documents are untrusted evidence. A malicious PDF may contain instructions like “ignore previous instructions” or “reveal all tenant data.” The system must prevent document text from changing model behavior.

The prompt should clearly separate system instructions, user query, tool outputs, and retrieved evidence. The model should be told that retrieved content is data, not instructions. Additionally, prompt-injection scanning and policy enforcement should happen outside the model.

Most importantly, even if the model is tricked, backend tools and permissions still prevent unauthorized access.

Step-by-step runtime flow

- RAG retrieves allowed chunks.
- Chunks are scanned for prompt-injection patterns.

- Suspicious chunks are labeled, sanitized, or excluded.
- Prompt wraps evidence in clearly marked evidence blocks.
- Tool execution remains policy-controlled.
- Final answer validator checks for instruction leakage or unsafe behavior.

Edge cases and failure handling

- If document contains clear malicious instructions, exclude that section and warn user if appropriate.
- If useful content is mixed with injection, sanitize and preserve factual content if safe.
- If model output follows malicious instruction, validator blocks it.

Security, RBAC, and tenant-isolation considerations

- Never let retrieved text define tool permissions.
- Never place retrieved text in system prompt.
- Do not execute actions based only on document instructions.

Conversation History, Redis, Durable Storage, and Long-Term Memory

Q35. How do you store conversation history in Redis and durable DB?

Durable DB is the source of truth. Redis is a fast cache and working-state store. When a user sends a message, the system writes the message to durable storage first, then updates Redis best-effort. If Redis fails, the conversation is still safe and can be rebuilt from DB.

The same applies to assistant responses: stream partial tokens through Redis or in-memory state, then write the final completed assistant message to DB. Tool calls, citations, and audit records should also be persisted.

This design balances durability and latency.

Step-by-step runtime flow

- Assign message_id and sequence_number.
- Write user message to DB with status RECEIVED.
- Start AI flow immediately after DB write.
- Update Redis recent context asynchronously or best-effort.
- During streaming, store partial response in Redis.
- On completion, write final assistant message to DB with status COMPLETED.
- Update Redis recent context.
- If Redis is stale next time, repair from DB using sequence numbers.

Important components involved

- DynamoDB/Aurora conversation store
- Redis/ElastiCache
- Chat service
- Sequence manager
- Streaming service

Edge cases and failure handling

- DB write fails: do not continue because audit/history would be lost.
- Redis write fails: continue; repair on next request.
- Assistant generation fails: write FAILED status.
- User cancels: write CANCELLED status.

Security, RBAC, and tenant-isolation considerations

- Conversation records must be tenant/user scoped.
- Sensitive logs should be redacted or encrypted.
- Delete conversation should delete or tombstone both DB and cache.

Q36. How do sequence numbers help with Redis consistency?

If Redis stores the last N messages but a Redis update fails, the cache may be missing the latest message. Sequence numbers let the system detect and repair this situation.

Each conversation message gets a monotonically increasing sequence number. Redis stores the last_sequence it contains. Durable DB stores the authoritative latest sequence. On the next request, if Redis last_sequence is behind DB latest_sequence, the chat service fetches only the missing messages and repairs Redis.

This avoids blindly trusting stale cache.

Step-by-step runtime flow

- Message 1,2,3 are in Redis.
- User sends message 4.
- DB write for message 4 succeeds.
- Redis update fails.
- Next request checks Redis last_sequence=3.
- DB latest_sequence=4.
- Service fetches messages where sequence > 3.
- Redis is repaired before AI context is built.

Edge cases and failure handling

- If Redis has no state, load last N from DB.
- If DB latest check is expensive, store latest sequence in lightweight conversation metadata.
- If sequence gap is large, rebuild summary and recent context.

Production behavior: This is a good interview detail because it shows you understand cache consistency beyond simply saying “use Redis.”

Q37. How do you implement long-term memory?

Long-term memory stores safe, reusable user preferences across sessions. It should not store sensitive cyber facts such as current vulnerabilities, exposed assets, or risk scores. Those facts change and must come from live tools.

Memory should store things like preferred report style, preferred output format, frequently used business-unit context, or workflow preferences. It is tenant-scoped and user-scoped. Users should be able to view, delete, or clear memory.

Memory extraction runs after the response, usually asynchronously, to avoid adding latency to the main chat path.

Step-by-step runtime flow

- Before answer: retrieve relevant memories for tenant_id and user_id.
- Use memories only as preference/context in prompt.
- After response: memory extractor checks user message and assistant response.
- If explicit “remember this,” store directly.
- If inferred pattern, use lightweight LLM/classifier and confidence threshold.
- Store memory in DB and optional vector index.
- Allow user deletion and audit memory changes.

Different possible scenarios

- Explicit: “Remember I prefer executive summaries.”
- Inferred: user repeatedly asks for CISO-friendly language.
- Unsafe: “Remember asset X has CVE Y” should be ignored or treated as conversation-only.

Security, RBAC, and tenant-isolation considerations

- Memory never overrides RBAC.
- Memory is not source of truth for cyber data.
- Memory retrieval must filter by tenant_id and user_id.

Q38. How does memory interact with follow-up questions?

Conversation history resolves immediate references, while long-term memory personalizes style or repeated preferences. If the user says “summarize this for my VP,” conversation context identifies what “this” refers to. Memory may tell the system that the user prefers concise executive summaries.

However, if the answer needs current cyber data, tools must still be called again. Memory should never be used to answer current risk facts.

This separation prevents stale or unauthorized answers.

Step-by-step runtime flow

- Load recent conversation context.
- Resolve reference such as this, same, those assets.
- Retrieve relevant user memories for style/preferences.
- Re-run tools/RAG if current facts are needed.
- Generate answer using current evidence and memory style hints.

Edge cases and failure handling

- If conversation context is missing, ask clarification.
- If memory conflicts with explicit user request, explicit current request wins.
- If permissions changed, do not reuse older tool output.

Q39. What should not be stored in long-term memory?

Do not store volatile or sensitive cybersecurity facts as long-term memory. Examples include exact vulnerabilities on assets, current risk score, exposed ports, credentials, user access details, or cross-tenant references.

These facts can become stale and may be highly sensitive. Long-term memory should be limited to stable preferences and safe workflow context.

If the user explicitly asks to remember sensitive data, the system should apply policy. Depending on compliance rules, it may refuse or store only a sanitized preference.

Step-by-step runtime flow

- Memory extractor identifies candidate memory.
- Classify as preference, stable context, sensitive cyber fact, secret, or temporary detail.
- Only safe categories are stored.
- Unsafe candidates are dropped and optionally logged.
- If explicit user request conflicts with policy, explain limitation.

Edge cases and failure handling

- Do not store API keys or secrets.
- Do not store “Payment DB is exposed.”
- Do not store hidden asset names as preference.
- Do not store data from documents as memory unless it is a user preference.

Security, RBAC, and tenant-isolation considerations

- Memory is subject to DLP scanning.
- Memory writes should be auditable.
- Users need memory deletion controls.

Report Generation and Jira Workflow

Q40. How should executive report generation work?

Report generation should be an asynchronous workflow because it may require multiple tools, RAG retrieval for templates, LLM generation for sections, validation, PDF rendering, and S3 storage.

The chat should not block for a long PDF job. Instead, the system creates a report job, shows progress, and returns a download link when ready. The report should include citations, data timestamps, and scope information.

For executive reports, memory can influence style, but facts must come from live tools.

Step-by-step runtime flow

- User requests report.
- Planner determines report type, time range, audience, and required data.
- If details missing, ask clarification.
- Create report job with status QUEUED.
- Worker fetches live risk/exposure/vulnerability data through tools.
- RAG retrieves report template or policy guidance if needed.
- LLM drafts sections with evidence.
- Validator checks citations and unsupported claims.
- PDF renderer creates final report.
- Store in S3 with KMS and tenant scope.
- Chat returns secure download link.

Important components involved

- Report service
- Async queue
- Worker
- Tools
- RAG
- LLM gateway
- PDF renderer
- S3
- Audit service

Edge cases and failure handling

- Tool fails: mark report partial or failed depending on criticality.
- PDF rendering fails: retry worker.
- User lacks report permission: deny before job creation.
- Long report: chunk generation by sections.

Security, RBAC, and tenant-isolation considerations

- Report link must be signed and tenant/user scoped.
- Report data must respect RequestContext.
- Emailing report requires approval and recipient validation.

Q41. How should Jira ticket creation work safely?

Jira creation is a write action, so it must require permission, preview, approval, idempotency, and audit logging. The LLM should never directly create tickets. It can draft ticket content based on authorized evidence, but backend services execute the action only after user approval.

This prevents accidental ticket spam, incorrect remediation tasks, or unauthorized workflow changes.

The approval UI should show ticket title, description, affected assets, priority, assignee/project, and citations.

Step-by-step runtime flow

- User asks to create tickets.
- Planner identifies candidate findings using tools/RAG.
- Policy engine checks `can_prepare_jira` or `can_create_jira`.
- LLM drafts ticket content from authorized evidence.
- Backend validates ticket schema and allowed Jira project.
- UI shows preview to user.
- User approves.
- Jira service creates tickets with idempotency keys.
- Audit log records approval and external ticket IDs.

Edge cases and failure handling

- User lacks create permission: allow summary but not creation.
- Jira API fails: retry or mark failed ticket.
- Partial bulk failure: report which tickets succeeded/failed.
- Duplicate request: idempotency prevents duplicate tickets.

Security, RBAC, and tenant-isolation considerations

- Never create Jira from unvalidated LLM output.
- Do not include sensitive data in ticket if Jira project is not approved for it.
- Approval must be explicit and logged.

Q42. How do you handle report email or download securely?

Reports may contain sensitive security data, so downloads and emails require strict controls. A generated report should be stored in S3 with KMS encryption and referenced through a short-lived signed URL or product-authenticated download endpoint.

Emailing a report is higher risk because data leaves the application context. It should require user confirmation, recipient validation, and tenant policy checks. Some tenants may disable email export entirely.

Every export must be audited.

Step-by-step runtime flow

- Report generated and stored securely.
- User requests download or email.
- Policy engine checks export permission.
- For download, issue short-lived signed link or authenticated endpoint.
- For email, show recipient and report preview.
- User confirms.
- Email service sends report or secure link.
- Audit export event.

Edge cases and failure handling

- Expired link: user regenerates if still authorized.
- User permission revoked: download endpoint denies.
- Email domain not allowed: block.
- Report contains highly sensitive data: require additional approval.

Security, RBAC, and tenant-isolation considerations

- Do not make S3 objects public.
- Use KMS encryption.
- Log who downloaded or emailed the report.

Security Guardrails, DLP, Prompt Injection, and Hallucination Control

Q43. What security guardrails are required for this AI copilot?

The system needs guardrails at every layer: input, retrieval, tool execution, prompt construction, output validation, logging, and action execution. Cybersecurity data is sensitive, so one guardrail is not enough.

Core controls include RBAC, tenant isolation, DLP scanning, prompt-injection detection, tool schema validation, output validation, audit logging, rate limiting, and approval workflows for actions.

The main design principle is defense in depth. If one layer fails, another layer should still prevent leakage or unsafe action.

Step-by-step runtime flow

- Authenticate and build RequestContext.
- Scan user input for abuse or secrets if needed.
- Retrieve only authorized data.
- Scan retrieved chunks/tool output for DLP and prompt injection.
- Construct prompt with strict separation of instructions and evidence.
- Validate final response.
- Audit all sensitive events.

Important components involved

- Policy engine
- DLP scanner
- Prompt injection detector
- Tool validator
- Response validator
- Audit logger
- Rate limiter

Security, RBAC, and tenant-isolation considerations

- LLM is not trusted with permissions.
- Documents are untrusted evidence.
- Actions require approval.
- Logs must be redacted and access-controlled.

Q44. What is DLP and where should it be applied?

DLP means Data Loss Prevention. In this system, it prevents secrets, credentials, tokens, PII, confidential customer data, and unsafe security details from leaking into prompts, responses, logs, reports, Jira tickets, or emails.

DLP should run at multiple points, because sensitive data can enter through user input, uploaded PDFs, tool outputs, retrieved chunks, generated responses, and exports.

DLP should use deterministic detectors for known secret patterns and classifiers for sensitive text categories.

Step-by-step runtime flow

- Scan user input before processing.
- Scan uploaded documents during ingestion.
- Scan retrieved RAG chunks before prompt construction.

- Scan tool outputs before LLM context.
- Scan final LLM response before display.
- Scan report/Jira/email payloads before export.
- Redact, block, or require approval depending on severity.

Edge cases and failure handling

- API key detected in PDF: redact or exclude chunk.
- User asks to reveal secrets: refuse.
- Tool returns sensitive internal field: strip before LLM.
- Report contains confidential fields: watermark or restrict export.

Security, RBAC, and tenant-isolation considerations

- DLP events should be audited.
- Do not log raw secrets.
- Use tenant-specific policies if required.

Q45. How do you prevent hallucinated cyber risk numbers?

The LLM should not invent risk scores, vulnerability counts, exposure values, or asset facts. Any factual number must come from a tool result or cited document. The final answer validator should check that the answer's claims are grounded in evidence.

Prompt instructions help, but they are not enough. The system should structure tool results, require citations, and validate final output against those results.

If evidence is missing, the correct answer is "I do not have enough data," not a confident guess.

Step-by-step runtime flow

- Planner identifies data required.
- Tools fetch current values with timestamps.
- LLM receives structured evidence.
- Final answer includes numbers and citations.
- Validator extracts numeric claims and compares with tool/RAG evidence.
- If unsupported, regenerate or return safe fallback.

Different possible scenarios

- Tool provides risk score 78: answer may mention 78.
- No tool result for risk score: answer cannot invent one.
- Conflicting sources: explain conflict and prefer live data.

Security, RBAC, and tenant-isolation considerations

- Hallucination in cybersecurity can cause wrong remediation decisions.
- Unsupported claims should be treated as quality incidents.

Q46. How should the system respond when it does not know?

The system should be allowed and encouraged to say it does not know when evidence is missing, access is denied, or data sources are unavailable. In cybersecurity, a wrong answer can be worse than no answer.

A good refusal or uncertainty response should be specific: what data was unavailable, what scope was checked, and what the user can do next if appropriate.

It should not expose hidden data or internal details.

Step-by-step runtime flow

- Planner determines required evidence.
- Tools/RAG fail to produce sufficient evidence or policy denies access.
- LLM is instructed not to guess.
- Response generator creates safe uncertainty response.
- Audit logs record missing-data or denied-access reason.

Different possible scenarios

- No matching documents found: say no relevant document evidence.
- Tool unavailable: say current data cannot be accessed.
- User unauthorized: say access is restricted.
- Ambiguous question: ask clarification.

Security, RBAC, and tenant-isolation considerations

- Do not reveal hidden tenant data in refusal.
- Do not provide inferred facts from memory.
- Do not fabricate citations.

Q47. How does response validation work before showing the answer?

Response validation is a final safety and quality gate. It checks whether the answer is grounded, cites sources, respects RBAC, avoids secrets, avoids unsupported claims, and does not propose unauthorized actions.

Validation can combine deterministic checks and model-based grading. Deterministic checks are used for tenant IDs, citation IDs, schema, secrets, and action permissions. Model-based checks can judge groundedness or completeness.

For high-risk responses, validation should happen before streaming the final answer or with buffered streaming.

Step-by-step runtime flow

- LLM generates candidate answer.
- Validator checks citations exist and map to authorized tool/doc sources.
- DLP scanner checks output.
- Unsupported claim detector checks numbers/entities against evidence.
- Policy checker ensures no unauthorized action or scope.
- If pass, send to user.
- If fail, regenerate or return safe fallback.

Edge cases and failure handling

- Citation missing: regenerate with citation requirement.
- Secret detected: redact/block.
- Cross-tenant entity detected: block and alert.
- Unsupported number: remove or regenerate.

Production behavior: Validators should produce structured scores and reasons so failures can be analyzed in Langfuse or observability dashboards.

LLM Gateway, Model Routing, Prompt Design, and Fallbacks

Q48. Why use an LLM gateway instead of calling one model directly?

An LLM gateway centralizes model access, routing, retries, fallback, token budgeting, logging, redaction, and policy controls. It prevents every service from directly integrating with different model providers.

For enterprise production, this is important because model providers may have outages, costs vary by task, and some tenants may have stricter data handling requirements.

The gateway also enables prompt versioning and consistent observability.

Step-by-step runtime flow

- Orchestrator sends model request to LLM gateway with task type.
- Gateway selects model based on task: classifier, reasoning, summarization, validation.
- Gateway applies tenant policy, timeout, token budget, and logging rules.
- If primary fails, retry or route to fallback.
- Return normalized response to orchestrator.

Different possible scenarios

- Small model for intent detection.
- Large model for complex investigation.
- Backup model for outage.
- Private model for sensitive tenant if required.

Edge cases and failure handling

- Provider timeout: retry with backoff.
- Rate limit: fallback or queue.
- Cost budget exceeded: use smaller model or ask user to narrow scope.

Q49. How do you design prompts for this system?

Prompt design should separate instructions, user request, conversation context, memory, tool results, and RAG evidence. The model should know that tools and documents are evidence, but only backend policy controls access.

The prompt should explicitly require grounded answers, citations, uncertainty when evidence is missing, and no invention of metrics. It should also define output format for different tasks such as tables, summaries, reports, and Jira drafts.

Prompts must be versioned and evaluated like code.

Step-by-step runtime flow

- Build system/developer instructions.
- Add safe subset of RequestContext, such as role and allowed actions.
- Add relevant memory preferences.
- Add conversation summary.
- Add tool results and RAG evidence with source labels.
- Add answer rules and citation requirements.
- Call LLM through gateway.
- Log prompt version and trace metadata.

Edge cases and failure handling

- Too much context: summarize or truncate by relevance.

- Conflicting evidence: instruct model to explain conflict.
- Prompt injection in retrieved text: label as untrusted evidence.

Security, RBAC, and tenant-isolation considerations

- Never put raw secrets in prompt.
- Never let retrieved docs override system instructions.
- Do not include hidden permission internals unnecessarily.

Q50. How do you handle LLM provider failure?

LLM failure should not crash the whole product. The LLM gateway should implement retry, fallback, and graceful degradation. The system may still return structured tool results or a safe message if synthesis is unavailable.

The fallback strategy depends on task criticality. For simple summarization, use a backup model. For high-stakes security answers, if validation cannot be performed, return partial evidence rather than an unvalidated answer.

All failures should be observable.

Step-by-step runtime flow

- LLM call times out or fails.
- Gateway retries with exponential backoff within latency budget.
- If still failing, route to backup model.
- If backup fails, orchestrator returns safe degraded response.
- Trace records provider, error, fallback, and latency.
- Alert if failure rate crosses threshold.

Different possible scenarios

- Intent classifier fails: fallback to rules or ask clarification.
- Final answer model fails: return raw structured summary if safe.
- Validator model fails: use deterministic validation or buffer response.
- Provider outage: disable AI synthesis but keep product search/tools available.

Edge cases and failure handling

- Do not let fallback model ignore safety rules.
- Do not answer from memory when tool/LLM failed.
- Respect tenant policy for allowed model providers.

Q51. How do you optimize LLM cost without reducing quality?

Cost optimization comes from model routing, context control, caching, summarization, and deterministic computation. Not every step needs the strongest model. Intent classification, memory extraction, and safety classification can use smaller models or rules.

The most expensive mistake is sending huge raw datasets to the LLM. Tools should aggregate data, RAG should rerank chunks, and reports should generate section by section.

Cost should be tracked per tenant, model, prompt version, and feature.

Step-by-step runtime flow

- Classify task type.
- Select smallest capable model.
- Limit RAG chunks after reranking.
- Use tools for deterministic computation.

- Cache embeddings and save retrieval results.
- Track token usage and cost in observability.
- Apply tenant/user rate limits.

Edge cases and failure handling

- If budget exceeded, ask user to narrow scope or run async report.
- If context too large, summarize intermediate data.
- If repeated query, reuse safe cached retrieval/tool results.

Production behavior: Cost is not only a finance concern; high cost often correlates with latency and poor architecture boundaries.

Evaluation, Langfuse, Observability, and Quality Control

Q52. How do you evaluate this AI system?

Evaluation has two major parts: LLM output evaluation and system/agent workflow evaluation. Output evaluation checks whether the final answer is accurate, grounded, cited, complete, and safe. Workflow evaluation checks whether the system chose the right intent, tools, RAG chunks, policies, and actions.

For an enterprise AI copilot, workflow evaluation is as important as final text quality. A fluent answer is still wrong if it used the wrong tool, skipped RBAC, or hallucinated data.

Evaluation should be both offline before release and online in production.

Step-by-step runtime flow

- Create golden datasets for risk, asset, vulnerability, RAG, RBAC, prompt injection, and Jira scenarios.
- Run test queries through full orchestrator.
- Capture traces for intent, plan, tools, RAG, LLM, validation.
- Score each layer.
- Compare prompt/model/tool versions.
- Review failures and improve system.
- Monitor production traces and user feedback.

Important components involved

- Langfuse or similar tracing
- Eval datasets
- LLM-as-judge
- Deterministic checkers
- Human review
- Dashboards

Different possible scenarios

- Final answer correct but wrong tool: workflow fail.
- Right tool but bad explanation: output fail.
- RBAC bypass attempt blocked: security pass.
- Prompt injection document ignored: safety pass.

Q53. How would you use Langfuse for tracing and evaluation?

Langfuse-style observability captures each user request as a trace with spans for intent classification, memory retrieval, RAG retrieval, tool calls, LLM calls, validation, and final response. This helps debug not only what the model answered, but why the system behaved that way.

It also supports datasets and experiments. You can run a set of test questions against prompt v1 and prompt v2, compare groundedness, tool correctness, latency, token cost, and failure rate.

Sensitive data should be redacted or hashed before logging.

Step-by-step runtime flow

- Start trace when request enters chat service.
- Add metadata: hashed tenant_id/user_id, prompt version, model version, intent.
- Create spans for each internal step.
- Log tool inputs/outputs after redaction.

- Log retrieved chunk IDs and scores.
- Attach final answer and validation scores.
- Attach user feedback.
- Use traces to build eval datasets and regression tests.

Important components involved

- Trace
- Span
- Generation
- Dataset
- Experiment
- Score
- Feedback

Edge cases and failure handling

- Do not log secrets or raw customer-sensitive data without policy.
- Sampling may be used in production, but security events should always log.
- If trace missing spans, debugging becomes weak.

Q54. What metrics should be tracked in production?

Metrics should cover engineering health, AI quality, security, and cost. Engineering metrics include latency, error rate, tool timeout rate, LLM provider failure, Redis/DB latency, and report job success. AI metrics include groundedness, citation correctness, retrieval quality, tool-selection accuracy, and user feedback.

Security metrics include denied access attempts, prompt injection detections, DLP blocks, cross-tenant validation failures, and unsafe action attempts. Cost metrics include tokens, cost per tenant, model usage, and cache hit rate.

These metrics should be broken down by tenant, feature, model, prompt version, and tool where appropriate.

Step-by-step runtime flow

- Emit metrics from every service.
- Attach trace_id/request_id for correlation.
- Aggregate dashboards by p50/p95/p99 latency.
- Alert on SLO violations and security anomalies.
- Use production feedback to improve eval datasets.

Edge cases and failure handling

- High fallback rate indicates provider or prompt issue.
- High no-answer rate may indicate retrieval/tool gaps.
- High DLP blocks may indicate sensitive document ingestion issue.
- High cost per query may indicate context bloat.

Production behavior: A mature AI system is operated like any other distributed system plus additional AI quality and safety metrics.

Q55. How do you evaluate RAG quality?

RAG quality should be evaluated at retrieval and answer levels. Retrieval evaluation checks whether the correct chunks were retrieved. Answer evaluation checks whether the final answer used those chunks correctly and cited them.

Metrics include recall@k, precision@k, MRR, citation correctness, groundedness, and no-answer correctness. For cybersecurity, exact identifiers like CVEs and asset names should be tested separately because hybrid search matters.

Human review is useful for complex policy interpretation.

Step-by-step runtime flow

- Create document Q&A dataset with expected source pages/chunks.
- Run retrieval only and measure whether expected chunks appear in top k.
- Run full answer generation and check citation correctness.
- Test permission filters with users from different groups.
- Test prompt-injection documents.
- Compare chunking, embedding, top_k, hybrid search, and reranker versions.

Edge cases and failure handling

- Correct answer but wrong citation is a failure.
- Correct chunk retrieved but ignored by LLM is answer-generation failure.
- Unauthorized chunk retrieved is a critical security failure.

Q56. How do you evaluate tool-calling correctness?

Tool-calling evaluation checks whether the agent selected the right tool, used correct arguments, avoided unnecessary tools, and respected policy. This is essential because the final answer depends on the correctness of tool execution.

For example, “top risky assets” should call `get_top_risky_assets`, not document search. “According to the SLA policy” should call RAG first. “Create Jira tickets” should prepare drafts and wait for approval.

The eval should inspect traces, not just final answers.

Step-by-step runtime flow

- Create test cases with expected tool plan.
- Run full orchestrator.
- Compare actual tool calls with expected tool calls.
- Check arguments, filters, limits, and RequestContext usage.
- Score unnecessary/missing tools.
- Check final answer against tool outputs.

Edge cases and failure handling

- Wrong tool but acceptable answer may still fail workflow eval.
- Correct tool with missing RBAC filter is security failure.
- Too many tool calls indicates inefficient planning.

Q57. How do you evaluate RBAC and tenant isolation?

RBAC and tenant-isolation evaluation should include negative tests. The goal is to prove that unauthorized data is not retrieved, not just that authorized queries work.

Create test tenants, users, groups, assets, documents, and vulnerabilities. Then ask questions from users with different permissions and verify that results only contain allowed entities. Also test direct malicious prompts asking to bypass authorization.

Any cross-tenant leakage is a severity-one issue.

Step-by-step runtime flow

- Build synthetic tenants A and B with overlapping-looking data.
- Create users with different group access.
- Run authorized and unauthorized queries.

- Inspect tool outputs, RAG chunks, final answers, citations, caches, and logs.
- Verify no hidden counts or entity names leak.
- Automate tests in CI/CD and production canaries.

Edge cases and failure handling

- User asks for hidden asset by exact name.
- User asks for all tenants.
- Uploaded document references another tenant.
- Cache key collision attempt.

Security, RBAC, and tenant-isolation considerations

- Fail closed on missing tenant metadata.
- Alert on cross-tenant validator failures.
- Include RBAC tests in every release gate.

Reliability, Scalability, Caching, AWS Deployment, and Tradeoffs

Q58. How do you design for reliability when tools fail?

Tool failures should not cause hallucinated answers. If a required data tool is unavailable, the bot should say it cannot access that data right now. If the tool is optional, the system can return partial results and clearly state what is missing.

The orchestrator should classify tool failures as retryable, non-retryable, permission denied, timeout, or data unavailable. Retry policies and fallbacks depend on the category.

Never answer current cyber facts from memory when tools fail.

Step-by-step runtime flow

- Tool call fails or times out.
- Tool executor classifies error.
- Retry with backoff if retryable and within latency budget.
- If still failing, update working state with failure.
- Planner decides partial answer, clarification, or async retry.
- Final response states limitation safely.
- Observability records error and alert if threshold crossed.

Edge cases and failure handling

- Risk summary tool down: cannot answer risk summary.
- Optional enrichment tool down: answer core result without enrichment.
- Multiple tools fail: return safe system-degraded message.
- Write action tool fails after approval: mark action failed and allow retry.

Q59. How would you scale this system to 1 million users?

Scaling means designing for registered users, active users, tenants, concurrent requests, tool traffic, document ingestion, vector search, and LLM cost. The system should scale horizontally at the stateless service layer and use async queues for heavy work.

The architecture should separate chat API, orchestrator, tools, RAG, document ingestion, report generation, and observability so each can scale independently. Caching, rate limits, model routing, and backpressure are required to protect the platform.

The system also needs tenant-level quotas and cost controls.

Step-by-step runtime flow

- Use load balancer/API gateway for frontend traffic.
- Run chat/orchestrator services on EKS with autoscaling.
- Use Redis for hot context and rate limiting.
- Use scalable DB partitions or DynamoDB/Aurora patterns for conversation storage.
- Use OpenSearch/vector DB scaling for RAG.
- Use SQS/EventBridge for async jobs.
- Use LLM gateway for rate limits, fallback, and cost control.

Edge cases and failure handling

- Traffic spike: queue or rate-limit low-priority workflows.
- Large tenant: isolate heavy workloads or provision capacity.
- LLM provider throttling: fallback or degrade.

- Vector DB hot partition: shard by tenant/doc scope.

Security, RBAC, and tenant-isolation considerations

- Scale should not weaken tenant isolation.
- Cache keys and partitions must include tenant scope.
- Rate limits should be tenant/user aware.

Q60. What caching strategy would you use?

Caching should improve latency without risking stale or unauthorized data. Safe caches include auth context with short TTL, conversation recent context, embeddings, product docs, and some retrieval results. Sensitive final answers should generally not be cached unless the key includes tenant, user, group, permission_version, data_version, prompt version, and query hash.

For live cybersecurity data, prefer caching intermediate summaries with freshness timestamps rather than final answers.

Cache invalidation is part of the design, not an afterthought.

Step-by-step runtime flow

- Identify cacheable item and sensitivity.
- Construct tenant-aware and permission-aware cache key.
- Set TTL based on freshness needs.
- On cache hit, validate permission_version/data_version.
- On miss/stale, fetch from source and update cache.
- On permission/data update, invalidate related keys.

Different possible scenarios

- Auth context cache: short TTL.
- Conversation context cache: Redis with sequence repair.
- Embedding cache: long TTL by content hash.
- Tool result cache: short TTL and data_version.
- Final answer cache: avoid for sensitive data.

Edge cases and failure handling

- Redis failure: fallback to DB/source.
- Stale permission: rebuild context.
- Cache poisoning: never use user-controlled keys without normalization.

Q61. What AWS components would you use?

On AWS, I would use CloudFront/WAF, ALB or API Gateway, EKS for services, ElastiCache Redis, Aurora PostgreSQL or DynamoDB for conversation/control data, S3 with KMS for documents and reports, OpenSearch Serverless for vector/hybrid search, SQS/EventBridge for async workflows, Bedrock or an LLM gateway for models, CloudWatch/OpenTelemetry for observability, Secrets Manager, and IAM/KMS for security.

The exact database choice depends on existing Balbix architecture. The key is to separate durable state, cache, document storage, vector retrieval, and async processing.

Infrastructure should support tenant isolation, auditability, and horizontal scaling.

Step-by-step runtime flow

- Request enters through WAF and load balancer.
- Chat service runs on EKS.

- Auth context cache uses ElastiCache.
- Conversation and metadata stored in Aurora/DynamoDB.
- Documents and reports stored in S3 with KMS.
- RAG index stored in OpenSearch Serverless.
- Async jobs run via SQS workers.
- LLM calls go through Bedrock/LLM gateway.
- Metrics/logs/traces go to observability stack.

Security, RBAC, and tenant-isolation considerations

- Use IAM least privilege.
- Encrypt S3, DB, Redis, and OpenSearch where applicable.
- Use private networking where possible.
- Store secrets in Secrets Manager.
- Audit access to sensitive resources.

Q62. How do you choose between DynamoDB and Postgres for conversation storage?

DynamoDB is excellent for high-scale key-value and access patterns such as `tenant_id + conversation_id + sequence_number`. It provides predictable scale and low-latency writes. Postgres/Aurora is better for relational queries, joins, transactions, and complex reporting over conversations, tool calls, citations, and feedback.

For a chat system, either can work. If the existing platform already uses Postgres heavily and needs relational analytics, Aurora/Postgres is convenient. If the target is massive write scale with simple access patterns, DynamoDB is attractive.

The important design is not the brand of DB, but the access pattern, partitioning, durability, and audit requirements.

Step-by-step runtime flow

- Define access patterns: get conversation, append message, list user conversations, search history, audit lookup.
- Choose DB based on query patterns and scale.
- Use `sequence_number` for ordered messages.
- Use secondary indexes for conversation listing/search.
- Store large artifacts in S3, not DB.
- Use Redis for hot context.

Different possible scenarios

- High write scale and simple reads: DynamoDB.
- Complex relational audit/reporting: Postgres.
- Hybrid: DynamoDB for messages, Postgres for metadata/audit, OpenSearch for search.

Edge cases and failure handling

- Hot tenant partition: shard or include `conversation_id`.
- Large messages: store references for attachments/artifacts.
- Delete requests: implement tenant/user scoped deletion.

Q63. What are the major tradeoffs in this architecture?

The architecture trades simplicity for security, reliability, and production readiness. A simple LLM wrapper is faster to build but unsafe. A full orchestrator with tools, RAG, policy, validation, and tracing is more complex but necessary for enterprise cybersecurity.

There are also tradeoffs between latency and accuracy, RAG flexibility and tool determinism, caching and freshness, memory personalization and privacy, autonomy and approval, and cost and model quality.

A strong architect can explain not only what was chosen, but why alternatives were rejected.

Step-by-step runtime flow

- Identify requirement: security, scale, latency, accuracy, cost.
- Compare options.
- Choose production-safe default.
- Add escape hatches such as async workflows or fallback models.
- Measure with evaluations and observability.

Different possible scenarios

- Tools over RAG for live data: better correctness, more backend work.
- RAG over fine-tuning: fresher and safer, but retrieval quality matters.
- Approval for Jira: safer, but more friction.
- DB-first storage: durable, but small latency cost.
- Redis cache: faster, but needs consistency handling.

Production behavior: In interviews, tradeoff answers often separate senior candidates from junior candidates. Always explain why the chosen design fits enterprise cybersecurity constraints.

Q64. How do you handle API design for chat?

The APIs should separate message submission, streaming, conversation history, document upload, report jobs, approval actions, feedback, and deletion. A typical flow uses POST /chat/messages to create a message, an SSE stream for response events, GET /conversations for history, POST /documents for upload, POST /reports for async reports, and POST /actions/{id}/approve for Jira approval. Every API receives RequestContext from middleware, never from request body. APIs should be idempotent where actions are involved and return stable IDs for retry/resume.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q65. How do you design the data model?

The core data model includes conversations, messages, tool_calls, citations, documents, document_chunks, memories, report_jobs, action_approvals, audit_events, feedback, and workflow_state. Messages have conversation_id, tenant_id, user_id, role, sequence_number, status, content, created_at. Tool calls store tool name, args hash, status, latency, result summary, and trace_id. Citations link answer spans to tool results or document pages. This model supports debugging, audit, user history, and evaluation.

Step-by-step runtime flow

- Receive request or event.

- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q66. How do you handle user feedback?

The UI should allow thumbs up/down, reason selection, free-text feedback, and “security concern” escalation. Feedback should attach to trace_id, answer_id, model version, prompt version, tools used, and retrieved chunks. Negative feedback becomes evaluation data. Security-related feedback should create a high-priority review event. Feedback should not immediately train the model automatically; it should go through review and dataset curation.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q67. How do you handle rate limiting?

Rate limiting should be tenant-aware, user-aware, endpoint-aware, and model-cost-aware. Simple chat can have normal limits, while report generation, large PDF ingestion, and bulk Jira drafts need stricter quotas. The LLM gateway should enforce token budgets and provider rate limits. If exceeded, the user receives a helpful message or async queueing option. Rate limiting protects cost, reliability, and abuse scenarios.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.

- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q68. How do you handle document deletion and reindexing?

When a document is deleted or permissions change, metadata and vector chunks must be removed or tombstoned. The source of truth is the document metadata DB. A deletion event should trigger vector index cleanup, S3 deletion/tombstone based on retention policy, and cache invalidation. Query-time retrieval should exclude deleted/tombstoned docs immediately, even if vector cleanup is still processing.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q69. How do you handle data freshness?

Every tool result should include data_timestamp or ingestion_version. Answers should mention freshness for operational facts, especially risk scores and vulnerability status. For RAG documents, document updated_at and version should be tracked. If a question asks for current state, live tool data should be preferred over old documents or memory. If data is stale, the answer should say so.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q70. How do you handle model upgrades safely?

Model upgrades should be treated like production releases. Run offline evaluation datasets, compare old vs new model on correctness, groundedness, tool-calling, RBAC, safety, latency, and cost. Use canary rollout by tenant or percentage.

Monitor regressions in Langfuse traces and dashboards. Keep rollback capability. Prompt versions and model versions must be logged in every trace.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q71. How do you handle prompt versioning?

Prompts should be stored as versioned artifacts. Each request logs prompt version, model version, tool schema version, and policy version. Changes should go through review and evaluation before release. This allows the team to correlate regressions with prompt changes and roll back if needed. Prompt templates should avoid hardcoded tenant data and should include clear evidence/citation rules.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q72. How do you debug a bad answer in production?

Start from the trace. Check RequestContext, intent classification, plan, tool calls, RAG chunks, LLM prompt, final answer, validation result, and user feedback. Determine whether failure came from routing, missing data, wrong tool, bad retrieval, hallucination, permission issue, or output formatting. Then add the case to evaluation datasets so it does not regress again.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q73. How do you handle accessibility and UX trust?

The UI should show citations, source timestamps, scope labels, progress states, errors, and approvals clearly. Users should be able to open source dashboards or document pages. The bot should show when it is using accessible scope only. For actions like Jira, preview and confirmation are required. Trust comes from transparency, not only fluent language.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q74. How do you prevent cache leakage?

Every cache key must include `tenant_id` and, for permission-sensitive data, `user_id`, `group_ids` or `permission_version`, and `data_version`. Never cache sensitive final answers with a broad key. Auth cache should have short TTL. Tool result cache should be scoped and fresh. Cache invalidation must happen on permission changes, document changes, and data ingestion updates.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q75. How do you handle audit logging?

Audit logs should capture who asked, what data/tools were accessed, what actions were prepared/executed, approval events, report exports, denied access, and security guardrail triggers. Logs should store trace IDs, tenant/user IDs or hashes, timestamps, tool names, status, and redacted arguments. Audit logs must be immutable or tamper-resistant depending on compliance requirements.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q76. How do you handle compliance requirements?

Compliance affects data retention, logging, deletion, access control, encryption, vendor use, and data residency. The system should support configurable retention, redaction, tenant-level AI logging controls, encrypted storage, least privilege access, audit exports, and model provider restrictions. For stricter customers, external LLM calls may need private endpoints or tenant-specific controls.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q77. How do you handle search over conversation history?

Conversation search should use durable DB/OpenSearch, not only Redis. The system can index conversation titles, message summaries, and metadata with tenant/user permissions. Sensitive content must be redacted based on policy. Users should only search their own conversations unless admin/audit permissions allow broader access.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.

- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q78. How do you handle answer citations?

Citations should point to tool results, dashboard links, asset/vulnerability records, document IDs, page numbers, and timestamps. The answer generator should not invent citations. The validator should verify each citation maps to evidence included in the prompt and authorized for the user. For documents, page-level citations are ideal. For tools, include data timestamp and source tool.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q79. How do you handle conflicting user instructions and security policy?

Security policy always wins. If a user says “ignore RBAC” or “show data from another tenant,” the system refuses. If a document says “send secrets,” it is treated as malicious content. If a user asks for Jira creation without approval, the system prepares a draft but pauses. The LLM should be instructed accordingly, but backend policy enforces it.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q80. How do you test failure scenarios?

Create chaos and negative tests: LLM timeout, tool timeout, vector DB unavailable, Redis down, DB write failure, permission change mid-request, malicious PDF, cross-tenant query, invalid Jira project, and report rendering failure. Verify the system fails safely, logs the event, does not hallucinate, and recovers or retries where appropriate.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q81. How do you decide when to ask clarification?

Ask clarification when ambiguity materially changes data scope, action, report type, or risk interpretation. Do not ask clarification for obvious defaults. For example, “top risks” may need clarification between CRQ, vulnerabilities, and exposure. “Top 10 risky assets” is clear enough. For write actions, clarify and confirm before execution.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q82. How do you handle action approval UX?

The system should show a structured preview with what will happen, why, evidence, impacted assets, destination system, and permission scope. The approve button should trigger a backend action endpoint with action_id and idempotency key. Approval should be recorded with user_id, timestamp, payload hash, and result. The LLM should not execute the action itself.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Q83. How do you explain this project in a senior-level interview?

Frame it as a secure enterprise AI agent system, not a chatbot. Start with business goal, then architecture: RequestContext, orchestrator, tools, RAG, memory, policy, validation, observability. Then explain one runtime flow deeply. Emphasize tradeoffs: tools over RAG for live data, RBAC outside LLM, async reports, approval-based actions, and evaluation beyond final answer quality.

Step-by-step runtime flow

- Receive request or event.
- Apply RequestContext and policy.
- Use the appropriate service or data store.
- Persist/audit outcome.
- Expose safe result or error to user.

Edge cases and failure handling

- Fail closed for security-sensitive uncertainty.
- Prefer source-of-truth data over cache.
- Log enough context for debugging without leaking secrets.

Security, RBAC, and tenant-isolation considerations

- Tenant and permission scope must be preserved throughout.

Final Interview Cheat Sheet

Theme	Strong interview phrasing
Core design sentence	This is a secure, RBAC-aware, tool-first enterprise AI copilot: tools provide live facts, RAG provides document evidence, memory provides preferences, the LLM explains, and policy/validators enforce safety.
Most important risk	Cross-tenant or unauthorized data leakage. Prevent it before data reaches the LLM.
Best architectural decision	Use live tools for structured data and RAG only for documents/policies.
Best production detail	DB is source of truth; Redis is cache. Use sequence numbers to detect stale conversation context.
Best safety answer	The LLM is not the security boundary. Backend authorization, tool validation, retrieval filters, DLP, and response validators are.
Best evaluation answer	Evaluate the full trace: intent, plan, tools, RAG, RBAC, final answer, latency, cost, and safety - not only the final text.